4COSC010C.2 Software Development II Coursework Deferral/Referral 23/24

Module Code / Title:	4COSC010C.2 / Software Development II
Assessment Component:	Coursework
Weighting:	50%
Qualifying mark:	30%
Academic Year:	2023 - 2024
Semester:	Deferral / Referral
Module Leader(s):	Dr. Ester Bonmati
Handed out:	Monday 20th June 2024
Due date:	**Monday 15th July 2024 at 1pm.** **You may be invited to a coursework demonstration.**
Learning outcomes:	LO1: Choose appropriate algorithms and data structures for problemsolving and implement these using a programming language. LO3: Develop solutions to common programming problems using classes to implement fundamental object-orientated concepts. LO4: Implement common data structures and data sorting algorithms. LO5: Undertake basic requirements gathering and data modelling exercises.
Expected deliverables:	You must submit the following files: - Self-evaluation form/ Project Report (word or pdf) - Your project exported into a Code.zip file. - Self-evaluation form/ Project Report (word or pdf) submitted to Turnitin link.
Method of submission:	Submission on Blackboard.
Marks	We will aim to give the marks in 15 working days, but this may not always be possible due to the large number of students in this module. All marks will remain provisional until formally agreed upon by an Assessment Board.
Feedback	Constructive feedback will be given with the marks.

Assessment regulations

Refer to section 4 of the "How you study" guide for undergraduate students to clarify how you are assessed, penalties and late submissions, what constitutes plagiarism, etc.

Penalty for late submissions

If you submit your coursework late but within 24 hours or one working day of the specified deadline, 10 marks will be deducted from the final mark, as a penalty for late submission, except for work which obtains a mark in the range 40 – 49%, in which case the mark will be capped at the pass mark (40%). If you submit your coursework more than 24 hours or more than one working day after the specified deadline you will be given a mark of zero for the work in question unless a claim of Mitigating Circumstances has been submitted and accepted as valid. It is recognised that on occasion, illness or a personal crisis can mean that you fail to submit a piece of work on time. In such cases you must inform the Student Centre in writing on a mitigating circumstances form, giving the reason for your late or non-submission. You must provide relevant documentary evidence with the form. This information will be reported to the relevant Assessment Board that will dec ide whether the mark of zero shall stand. For more detailed information regarding University Assessment Regulations, please refer to the following website:http://www.westminster.ac.uk/study/current-students/resources/academic-regulations

Coursework description

Important notes

- The use of generative AI (such as ChatGPT) for code generation is strictly **prohibited**. Generative AI can be used to: explain difficult concepts and find bugs in your code.
- Only use standard arrays and **do not use dynamic arrays** (e.g., ArrayList , Map, etc).
- Implement all the tasks in a single project.
- Use descriptive names for your variables and user-defined methods and add comments to explain your code, and use a good style.
- Implement and re-use your self-defined methods within your coursework when possible.
- You must **reference within your code** any code adapted from external or other sources, or any technology you may have used to implement your solution (e.g., ChatGPT).
- For each task, read first what is requested, think about the design (you can use pen + paper) and then implement it.
- You can use an Context

A new cinema company called 'The London Lumiere' has asked you to design and implement a new Java program to manage and control the seat reservations effectively. Your assignment involves designing and implementing this program. You will be working with the cinema's seating plan, which is shown below:

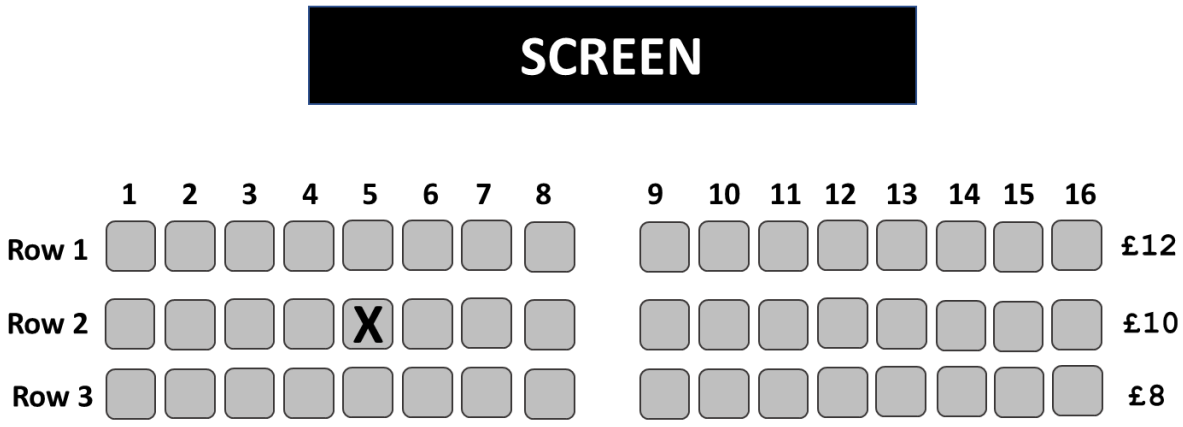


Figure 1. Cinema's seating plan. The row number is indicated at the front. The seat number is indicated at the top. The price for each seat is indicated at the end of the row (all seats in the same row have the same price). The seat marked with an 'X' corresponds to seat 5 in row 2. Seats start at 1 for each row. There is a gap between seats 8 and 9.

It is important that you follow the specifications shown in the picture above regarding the number of rows, seats, and seat price given for each row when implementing your coursework.

You must reference any external sources within your code.

Considering the picture above (Figure 1), complete the following tasks:

Part A: Main program (40 marks)

Task 1) Create a new project titled <your_student_id_number>_CinemaManagement (e.g., w123456_CinemaManagement). Your project should have a class file named CinemaManagement.java (or <your_student_id_number>_CinemaManagement.java).

This class should contain a **`main`** method that initiates the program by displaying the following message: 'Welcome to The London Lumiere'.

Implement the seat management system using standard arrays (**the use of dynamic arrays such as ArrayList or similar is prohibited, and their use will be penalised**) to keep track of the seats that have been sold and the seats that are still available. Use the value 0 to indicate a seat is available and the value 1 to indicate a seat has been sold.

At the program's start, all seats should be available (value of 0). The main method will be called when the program starts (entry point) for **all tasks** described in this coursework.

4 marks

Task 2) Add a user menu in your `main` method that displays the following menu and asks the user to select an option. Include data input validation in your menu implementation. After the user has selected an option, the menu should repeat and ask the user to select another option. Option '8' should terminate the program without crashing or giving an error. The rest of the options will be implemented in the next tasks.

```
-----------------------------------------------
Please select an option:
   1) Buy a ticket
   2) Cancel ticket
   3) See seating plan
   4) Find first seat available
   5) Print tickets information and total price
   6) Search ticket
   7) Sort tickets by price
   8) Exit
-----------------------------------------------
Select option:
```

Tip: Think carefully about which control structure you will use to decide what to do after the user selects an option (see Lecture in Variables and Control Structures).



Include a screenshot of your program's menu in the self-assessment form.

8 marks

Task 3) Create a method called **`buy_ticket`** that asks the user to input a row and seat numbers. Check that the row and seat numbers are correct and valid and that the seat is available. If the seat is available, and the numbers are valid, record the seat as occupied with value 1 and display the message 'The seat has been booked'. If the seat is

not available display the message 'This seat is not available'. Call this method when the user selects '1' in the main menu.

6 marks

Task 4) Create a method called **`cancel_ticket`** that marks a seat as available. It should ask the user to input a row and seat numbers. Check that the row and seat numbers are correct and valid and that the seat is unavailable. If the seat is unavailable, and the numbers are valid, record the seat as available with value 0 and display the message 'The seat has been cancelled'; otherwise, show the message 'This seat is already available'. Call this method when the user selects '2' in the main menu. Include a screenshot of your program's output in the self-assessment form.

6 marks

Task 5) A) Create a method called **`print_seating_area`** that shows the seats that have been sold and the seats that are still available, and the position of the screen. Display available seats with the character 'O' and the sold seats with 'X' following the format below. Call this method when the user selects '3' in the main menu.

The output should show the following when no tickets have been bought (start of the program):

```
*****************
*      SCREEN     *
*****************

OOOOOOOO OOOOOOOO

OOOOOOOO OOOOOOOO

OOOOOOOO OOOOOOOO
```

After purchasing a ticket for row 1, seat 6, and a ticket for row 3, seat 10, using option '1' in the menu twice, the output should display the following:

```
*****************
*      SCREEN     *
*****************

OOOOOXOO OOOOOOOO

OOOOOOOO OOOOOOOO

OOOOOOOO OXOOOOOO
```



Include a screenshot of your program's seating in the self-assessment form.

8 marks

Task 6) Create a method called **`find_first_available`** that finds the first seat which is still available. You should search in row 1 first, then row 2, etc. The output should display only one seat. Call this method when the user selects '4' in the main menu.

8 marks

Part B: Classes and Objects (45 marks)

Part B of coursework needs to be integrated with part A. Do not create a new project for part B. In part B, you will add two classes (Ticket and Person) in your project that store extra information when buying a seat.

Task 7) Create a new class file called **`Person`** (Person.java) with the following attributes: name, surname, and email. Add a constructor that takes the 3 variables as input to create an object Person. Add all the getters and setters for the class Person. Add a method that prints the information from Person.

4 marks

Task 8) Create a new class file called **`Ticket`** (Ticket.java) with the following attributes: row, seat, price, and Person. Person is an object created using the class `Person` from Task 7. Add all the getters and setters for the class ticket. Add a method that prints the information of a Ticket (including the information of the Person).

4 marks

Task 9) In the main program, add a standard array of type Tickets to store all the tickets sold in that session. Then:

1) Extend the **`buy_ticket`** method such that when buying a ticket, it asks for all the information of a Person, creates a new Ticket with the corresponding price and seat information and adds the ticket to the new array of Tickets.
2) Extend the **`cancel_ticket`** method such that when cancelling a ticket, it removes the ticket from the array list of tickets.

10 marks

Task 10) Create a method called **`print_tickets_info`** that prints the information of all tickets that have been sold during the session (including the person's details), and calculates the total price of the tickets sold during the session.

To calculate the total price of the tickets, see Figure 1. As example, after purchasing a ticket for row 1, seat 6, and a ticket for row 3, seat 10, the total price will be £20.

Call this method when the user selects '5' in the main menu.

9 marks

Task 11) Create a method called **`search_ticket`** that asks the user to input a row and seat numbers and checks if someone has bought that seat. If someone has bought the seat, use a search algorithm to search the ticket and print the Ticket and Person information; otherwise, it should display 'This seat is available'. Call this method when the user selects '6' in the main menu. ***9 marks***

Task 12) Create a method called **`sort_tickets`** that uses a sorting algorithm to sort the array of Tickets by price and prints all the tickets information (ascending order). Call this method when the user selects '7' in the main menu. ***9 marks***

Part C: Testing, style, and self-evaluation (15 marks)

Download the self-evaluation form from Blackboard and complete the following:

1) Self-evaluation form. ***5 marks.***
2) Testing part A. ***4 marks.***
3) Testing part B. ***4 marks.***

You will be evaluated on your coding style. Ensure that you use a good coding style in the submitted coursework to facilitate understanding: add comments to key parts of the code, use indentation, use informative names for variables and methods, create and reuse your selfdefined methods (to avoid repetition), etc. ***2 marks.***

Part D: Coursework demonstration

Based on the quality of your submission, you may need to attend a mandatory coursework demonstration. You will receive an email stating if you need to attend the demo. The demo will be on campus (room and date to be confirmed).

Failure to attend the demonstration will result in 0 marks for the coursework.

Marking scheme

The marking scheme of this coursework can be found in Blackboard under the coursework folder.

Submitting your coursework

1) Create a ZIP file of your project created based on the used IDE(i.e IntelliJ): Make sure the downloaded zip file is in working condition. You need to do the demonstration using downloaded zip file from Black Board during VIVA.

2) Rename your code.zip file and the self-assessment form in following format.
Code File : <IITNO>_<UoWNo>.zip
Self Assessment form : <IITNO>_<UoWNo>.pdf

3) Go to Blackboard's module page and select Assessment (coursework) > Coursework submission. **Attach below files as separate files in the same submission.**

i. Coursework report/ Self-assessment form (word or pdf)
ii. Code.zip file

4) Coursework report MUST be also submitted to Turnitin link provided separately.
Not submitting report to Turnitin Link will result in no VIVA slot and 0 marks

5) Submit your work.

END